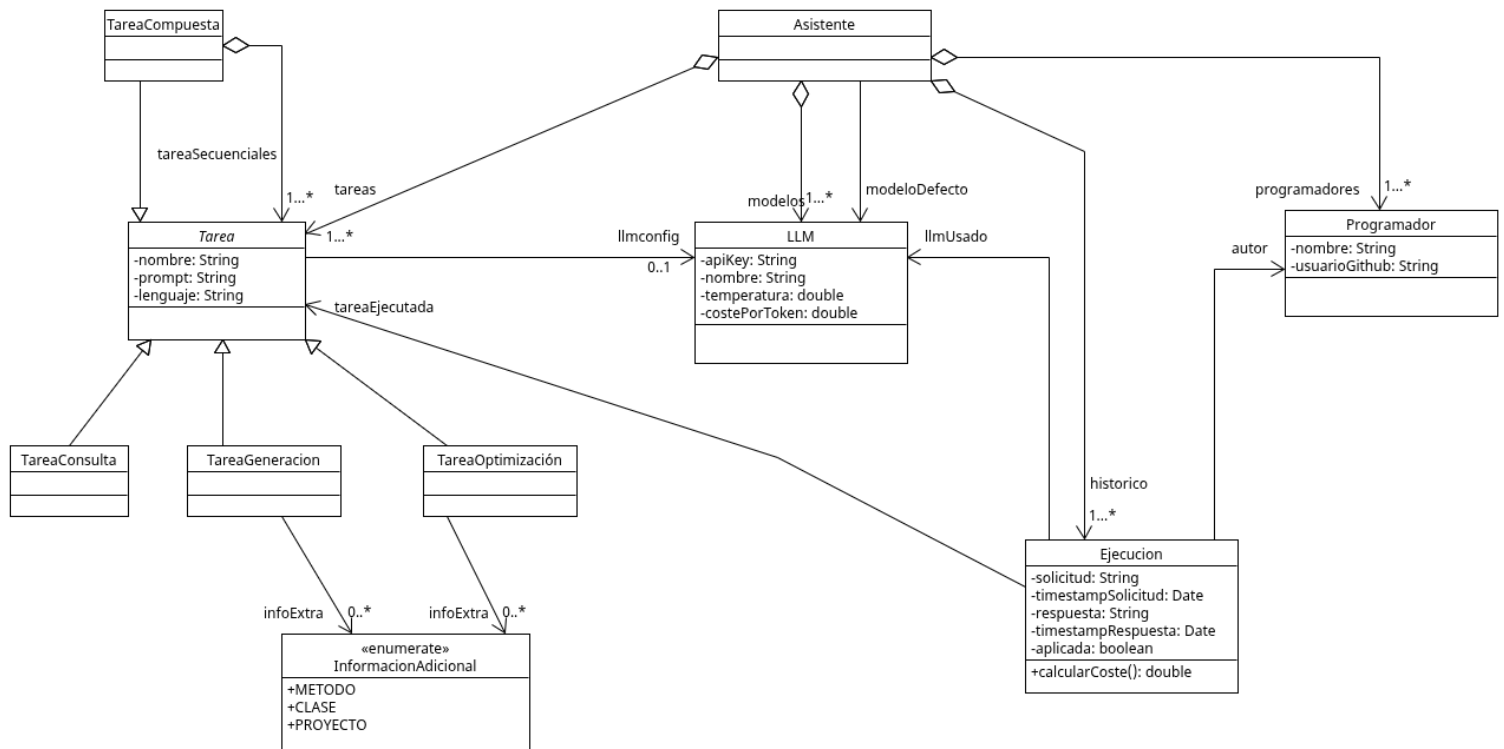


Apartado 2

a)



En este diagrama de clases podemos destacar varias cosas. Hemos propuesto un diseño en el que incluimos una clase padre que será **Tarea**, de la cual heredan otras clases como **TareaConsulta**, **TareaGeneracion**, **TareaOptimización** y **TareaCompuesta**, evitando así la repetición de mismos atributos en cada una de las clases dando una mayor claridad al diseño.

Esto a su vez, nos permite ahorrarnos métodos o atributos repetidos, ya que al guardar el histórico en el **Asistente** podemos calcular los costes totales con un solo método de forma genérica, ya que no necesitamos saber de manera específica qué tipo de tarea es la que se ha ejecutado.

Por último, también hemos creado una enumeración para la información adicional de las tareas ya que son especificadas en el enunciado de forma que están limitadas a solo tres opciones (método, clase o proyecto), por lo que ponerlo como un atributo de tipo **String** no nos parecía correcto. Además, hemos relacionado la **Ejecución** directamente con el **LLM** para asegurarnos de que el precio del historial no cambie si la tarea cambia de modelo en el futuro.

b)

1. Para obtener el coste de la ejecución, ya tenemos añadido el método `calcularCoste()` en la clase `Ejecucion`, ya que es este "ticket" el que guarda los textos de la solicitud y la respuesta, y además conoce el LLM exacto que se usó.
2. Para obtener la lista de tareas ordenadas por número de solicitudes, el método `getTareasPorSolicitudes()` lo pondríamos en la clase `Asistente`. Como el asistente es el que guarda todo el histórico, es el único que puede revisar cuántas veces se ha usado cada tarea. Para esto nos apoyaríamos en un método auxiliar privado `contarSolicitudes()`.
3. Para obtener las tareas aplicadas ordenadas, el método `getTareasAplicadas()` también iría en el `Asistente`. Para que esto funcione, en la clase `Ejecucion` necesitaríamos tener un método `esAplicada()` que nos devuelva si fue útil o no.
4. Para calcular el coste acumulado por programador, meteríamos el método `getCosteAcumulado(Programador p)` en el `Asistente`. Así puede recorrer su histórico, buscar qué ejecuciones son de ese programador en concreto, e ir sumando.

c)

```
public double calcularCoste() {  
    int tokensTotales = contarTokens(this.solicitud) +  
    contarTokens(this.respuesta);  
  
    return tokensTotales * this.llmUsado.getCostePorToken();  
}
```

```
public double getCosteAcumulado(Programador p) {  
    double costeTotal = 0.0;  
  
    for (Ejecucion ejecucion : this.historico) {  
        if (ejecucion.getAutor().equals(p)) {  
            costeTotal += ejecucion.calcularCoste();  
        }  
    }  
  
    return costeTotal;  
}
```